Analisador de Conexões de Aeroportos: Uma Abordagem Teórico-Prática baseada em Grafos Dirigidos, BFS e Dijkstra

Guilherme Belém 114031, Gabriel Fontes 112958, João Victor Camargo 113803
Engenharia da Computação

***Abstract*—Neste trabalho, apresentamos o desenvolvimento do *Analisador de Conexões de Aeroportos*, uma aplicação computacional baseada na Teoria dos Grafos voltada para a análise e otimização de rotas aéreas no Brasil. Modelamos a malha aérea nacional como um grafo dirigido e ponderado composto por 30 aeroportos (vértices) e 132 voos diretos (arestas), cobrindo todos os 26 estados brasileiros e o Distrito Federal. Para encontrar as melhores rotas sob diferentes critérios de decisão, implementamos os algoritmos de Busca em Largura (BFS) — para minimizar o número de escalas — e o Algoritmo de Dijkstra com fila de prioridade (*min-heap*) — para minimizar o tempo total de voo e o preço final da passagem. Além disso, criamos uma regra de negócio específica para aplicar descontos progressivos em viagens com conexões. Nossos testes comprovam a eficiência da modelagem por lista de adjacência e trazem uma comparação quantitativa e qualitativa do desempenho dos algoritmos aplicados ao transporte aéreo.**

***Index Terms*—Teoria dos Grafos, Grafo Dirigido, Busca em Largura (BFS), Algoritmo de Dijkstra, Malha Aérea.**

I. Introdução

No transporte aéreo, buscar rotas de forma eficiente exige responder a perguntas frequentes de conectividade e menor caminho: *Existe voo direto entre duas cidades? Qual trajeto tem o menor número de conexões? Qual opção sai mais barata ou mais rápida?*

Para resolver esse problema, usamos a Teoria dos Grafos como base matemática para representar toda a malha aérea brasileira, onde modelamos os aeroportos como vértices e os voos como arestas ponderadas. Neste artigo, detalhamos como pensamos, estruturamos e implementamos o *Analisador de Conexões de Aeroportos*, uma ferramenta visual e interativa que desenvolvemos para responder a essas dúvidas utilizando os algoritmos de Busca em Largura (BFS) e Dijkstra.

II. Modelagem do Grafo e Arquitetura

Modelamos a infraestrutura de voos por meio de um **Grafo Dirigido e Ponderado** $G = (V, E)$, no qual definimos:

- V como o conjunto de 30 aeroportos principais cobrindo as 27 unidades federativas do Brasil.
- E como o conjunto de 132 voos diretos entre esses aeroportos.

A. Decisões de Modelagem

- **Direcionamento:** Optamos por um grafo dirigido porque a existência de um voo no sentido $A \to B$ não garante que exista um voo $B \to A$ com os mesmos valores de preço ou duração.
- **Dois Pesos por Aresta:** Definimos dois pesos independentes para cada aresta: a *duração* (em minutos) e o *preço* (em reais). Dessa forma, conseguimos rodar o mesmo algoritmo de Dijkstra alterando apenas qual critério queremos otimizar.
- **Lista de Adjacência:** Como a malha aérea brasileira é esparsa, escolhemos representá-la por Lista de Adjacência em vez de Matriz de Adjacência. Isso reduziu o consumo de memória e agilizou a busca pelos vizinhos de cada aeroporto.

B. Topologia da Malha Aérea

Dividimos os 30 aeroportos da nossa base em três categorias:

1) **Hubs Principais (Vermelho):** Brasília (BSB) e Guarulhos (GRU), onde concentramos a maior parte das conexões.
2) **Sub-hubs Regionais (Laranja):** Aeroportos estratégicos como Confins (CNF), Recife (REC), Salvador (SSA) e Manaus (MAO).
3) **Aeroportos Regulares (Azul):** Demais capitais e polos regionais.

C. Arquitetura do Projeto

Organizamos a estrutura do projeto integrando base de dados, lógica dos algoritmos e interfaces de usuário:

- **Base de Dados:** Criamos o arquivo `aeroportos_dados.json` com as informações dos 30 aeroportos e 132 voos.
- **Grafo em Memória:** Carregamos os dados em estrutura de lista de adjacência.
- **Algoritmos:** Escrevemos os algoritmos BFS e Dijkstra em Python (`analisador_aeroportos.py`) para uso em terminal e em JavaScript (`algoritmos.js`) para a aplicação web.
- **Interface:** Desenvolvemos uma interface web (`index.html`, `style.css`, `app.js`) que permite comparar as rotas em cards e visualizar a animação da busca.

III. Algoritmos e Funcionalidades

A. Busca de Rota Direta

Implementamos uma função simples que verifica se há um voo direto da origem o até o destino d analisando apenas as arestas de saída da origem. A complexidade dessa consulta é de $\mathcal{O}(\text{grau}(o))$.

B. Busca em Largura (BFS) – Menos Escalas

Utilizamos a Busca em Largura quando o objetivo do usuário é encontrar o trajeto com menor quantidade de conexões, ignorando tempo total e custo. O algoritmo percorre a rede nível por nível utilizando uma fila (FIFO) com complexidade $\mathcal{O}(|V| + |E|)$.

Exemplo Prático (JPA → MAO): Na busca de João Pessoa para Manaus, a expansão por níveis ocorre da seguinte forma:

- Nível 0: JPA (Origem)
- Nível 1: REC
- Nível 2: GRU, VCP, BSB, SSA, NAT, MCZ, AJU, GIG
- Nível 3: MAO (encontrado ao expandir os vizinhos de GRU)

Neste caso, o BFS nos retornou a rota JPA → REC → GRU → MAO (2 escalas, R$ 1.431,00). Como o BFS foca no menor número de arestas e pega o primeiro nó do nível que encontra o destino, ele não avalia o custo financeiro (via BSB também seriam 2 escalas, mas por R$ 927,00).

C. Algoritmo de Dijkstra – Tempo e Preço

Para calcular a rota mais rápida ou a mais barata, aplicamos o Algoritmo de Dijkstra utilizando uma fila de prioridade (*min-heap*), o que nos garante uma complexidade de $\mathcal{O}((|V| + |E|) \log |V|)$.

Exemplo Prático (GRU → BEL pelo menor preço): Ao buscarmos a rota de Guarulhos para Belém por menor preço, durante o relaxamento dos nós, o algoritmo visita Brasília (BSB) com custo acumulado de R$ 320. A partir de BSB, a rota até BEL é atualizada para $320 + 410 =$ R$ 730, o que supera a opção do voo direto (R$ 890). Com o desconto de conexão aplicado, o valor final fica em R$ 693,50.

IV. Regra de Negócio: Desconto de Conexão

Inserimos uma regra de negócio realista no sistema: conceder 5% de desconto no preço total da passagem por cada escala realizada, até o limite de 15%. A equação da tarifa final ficou definida como:

$$P_{\text{final}} = P_{\text{total}} \times (1 - d) \tag{1}$$

onde o desconto d é dado por:

$$d = \min(\text{escalas} \times 0.05,\ 0.15) \tag{2}$$

Essa regra nos permitiu simular cenários reais onde voar com conexão é mais barato do que pegar um voo direto longo:

- **Voo Direto (GRU → BEL):** R$ 890,00 (195 min, 0 escalas).
- **Voo com 1 Escala (GRU → BSB → BEL):** R$ 693,50 (205 min, 5% de desconto).

V. Resultados e Análise Comparativa

Resumimos na Tabela I as características teóricas dos métodos que implementamos. Na Tabela II, apresentamos os resultados reais gerados pelo nosso sistema para diferentes pares de aeroportos.

TABLE I
Comparativo dos Algoritmos Implementados

Critério	Algoritmo	Complexidade	Otimização
Rota direta	Lista Adjacência	$\mathcal{O}(\text{grau}(v))$	Existência de aresta
Menos escalas	BFS	$\mathcal{O}(\|V\| + \|E\|)$	Nº de voos
Mais rápida	Dijkstra (Duração)	$\mathcal{O}((\|V\| + \|E\|) \log \|V\|)$	Soma do tempo
Mais barata	Dijkstra (Preço)	$\mathcal{O}((\|V\| + \|E\|) \log \|V\|)$	Soma do preço

TABLE II
Resultados Obtidos nos Testes do Nosso Sistema

Origem → Destino	Menos escalas (BFS)	Mais rápido	Mais barato
GRU → BEL	GRU-BEL (R$ 890)	GRU-BEL (3h15)	GRU-BSB-BE
JPA → MAO	JPA-REC-GRU-MAO (R$ 1.431,00)	JPA-REC-BSB-MAO (5h35)	JPA-REC-BS (R$ 927,00)
CWB → MCP	CWB-GRU-BSB-MCP (R$ 918,00)	CWB-CNF-BSB-MCP (5h05)	CWB-CNF-B (R$ 855,00)
FLN → PVH	FLN-GRU-BSB-PVH (R$ 936,00)	FLN-GRU-BSB-PVH (5h30)	FLN-POA-BS (R$ 900,00)

VI. Decisões do Grupo e Limitações

- **Busca por Cidade:** Pensamos no uso prático do usuário, que busca por cidade e não apenas por sigla. Por isso, quando o usuário seleciona ”São Paulo (todos)”, nosso sistema avalia automaticamente todas as combinações entre GRU, CGH e VCP para entregar o melhor resultado global.
- **Cálculo do Desconto no Dijkstra:** Como o Dijkstra realiza o relaxamento acumulando o preço bruto de cada trecho, a regra de desconto é calculada após a construção do caminho. Em casos pontuais, uma rota ligeiramente mais cara no bruto poderia se tornar mais barata após o desconto de 15%, algo que pretendemos evoluir em versões futuras.
- **Conjunto de Dados:** Utilizamos dados estimados de preços e durações com base na malha aérea real brasileira para fins de estudo e modelagem.

VII. Conclusão

Com a realização deste projeto, conseguimos aplicar na prática os conceitos da Teoria dos Grafos para resolver um problema real de transporte e logística. A implementação combinada de Lista de Adjacência, BFS e Dijkstra nos permitiu construir uma ferramenta rápida, dinâmica e eficiente. A visualização das rotas e a comparação direta entre os três critérios mostraram claramente como diferentes algoritmos respondem a diferentes prioridades de busca.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein, *Algoritmos: Teoria e Prática*, 3ª ed. Elsevier, 2012.

[2] J. A. Bondy e U. S. R. Murty, *Graph Theory with Applications*. Elsevier Science Publishing, 1976.